

Embedded C Workshop

Made by Matheus and Brady

Outline

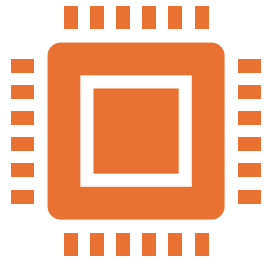
Disclaimer

My rant as an
EE student

The specifics
of Embedded
C

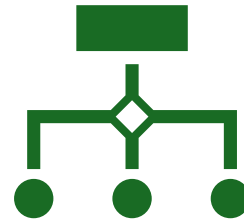
Live Demo
with Code
Walkthrough

Disclaimer



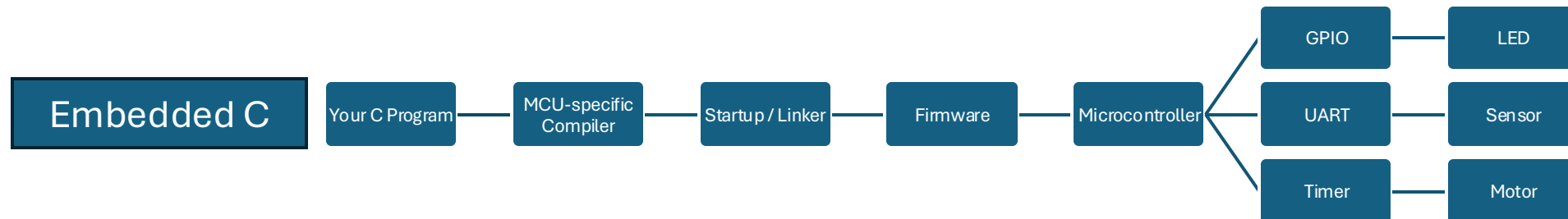
It is highly recommended that you use the appropriate IDE when programming for an Embedded System

Be it an Arduino, Raspberry Pi, ESP32, or other



Otherwise, you will need to do some extra steps to send your code to the microcontroller

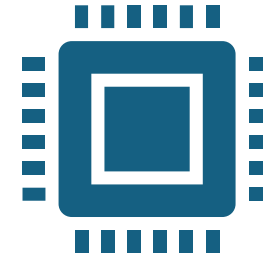
Embedded C is basically just C with extra steps



Choosing the Microcontroller



**You need to actually know what hardware
you're going to be programming on**



Such as:

ATmega328P

STM32F103

PIC16F877A

ESP32

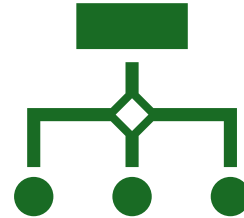
Raspberry PI

Arduino

Configuring the Microcontroller's Hardware



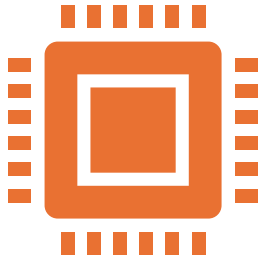
Instead of simply doing `printf("Hello World");`, we need to manually handle the actual hardware interaction



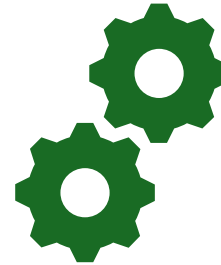
Could be configuring the GPIO registers to be an input or output

```
` DDRB |= (1 << PB0);`
```

Configuring Peripherals



You will need to configure the corresponding hardware registers/peripheral drivers for any peripherals you want to use



Using stuff like: GPIO, ADC, PWM, UART, SPI, I2C, Timers, and Interrupts

For a timer you would need to set several registers before the timer can actually do anything

Managing Limited Hardware Resources

Many small microcontrollers do not have a lot of hardware resources available compared to a modern PC

You have to pay more attention to stuff like RAM, Flash, stack size, heap usage, CPU cycles, power consumption, and timings

Flashing the Microcontroller

You will need to physically/programmatically transfer the firmware to the MCU

If using an IDE, typically built in

For an STM32 you might use an ST-Link

Debugging

You're no longer just looking at the code

When something goes wrong, it could be either hardware or firmware

You'll need some way to be able to debug the hardware as well as the firmware to actually find any problems in your embedded system

The Specifics of Embedded C

Hardware Registers

GPIO

Timers

Interrupts

General Takeaway

Hardware Registers: Controlling the MCU

-
- Special memory locations connected directly to hardware
 - Individual bits control specific hardware functions
 - Writing to a register changes the physical behavior of the MCU
 - Example:

...

```
DDRB |= (1 << PB0);
```

```
PORTB |= (1 << PB0);
```

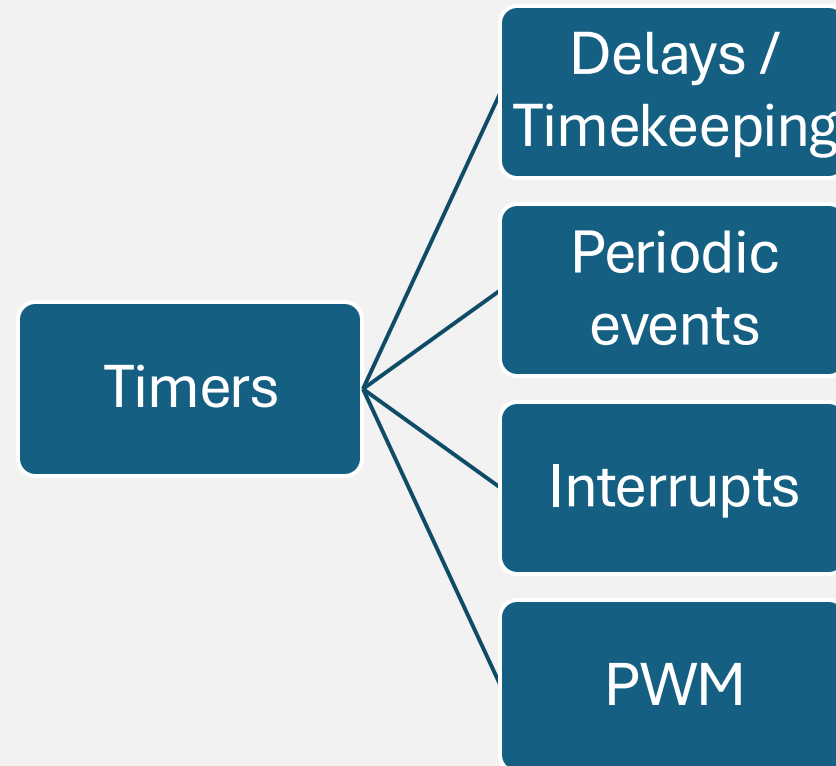
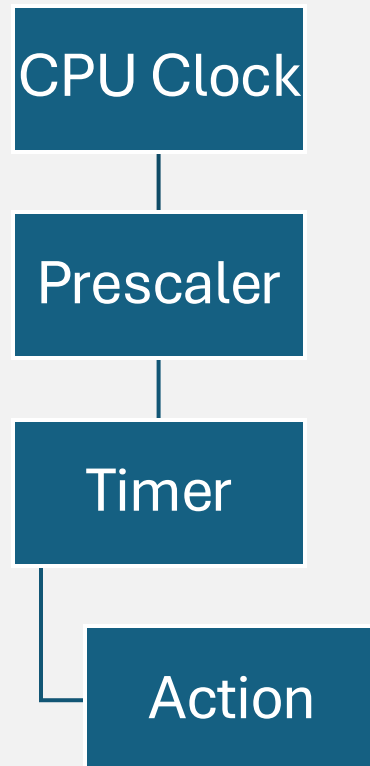
...

GPIO: Digital Inputs & Outputs

-
- GPIO pins allow the microcontroller to read external signals and control external devices
 - Output
 - ` ` `
 - `DDRB |= (1 << PB0); // Configure Pin as Output`
 - `PORTB |= (1 << PB0); // Set Pin as HIGH/On`
 - ` ` `
 - Input
 - ` ` `
 - `DDRB &= ~(1 << PB1);`
 - `If (PINB & (1 << PB1)) { // When Pin is HIGH/On, if function will process }`
 - ` ` `

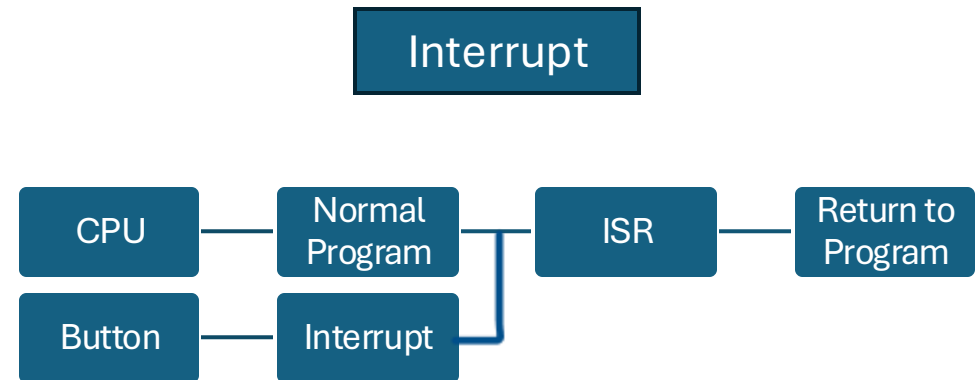
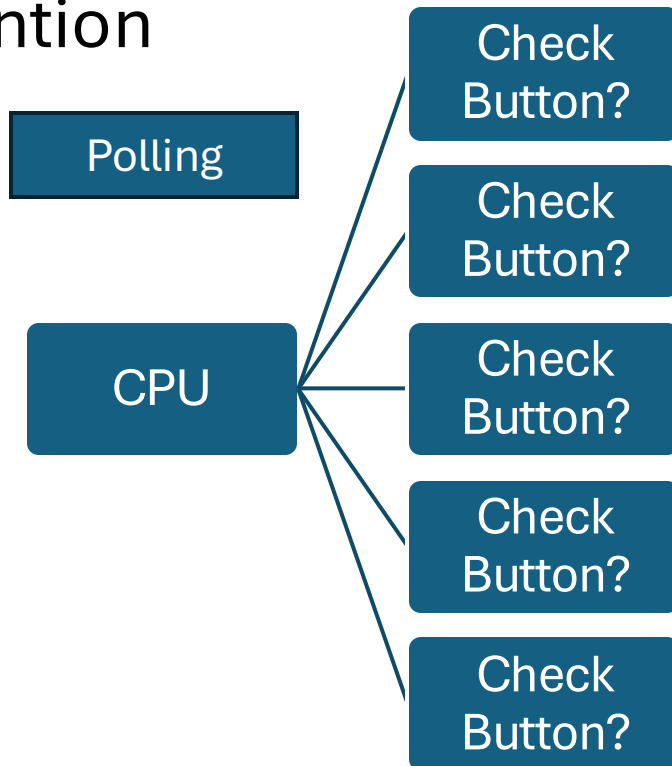
Timers: Hardware-Based Timing

- A hardware counter that counts clock cycles without the CPU constantly counting

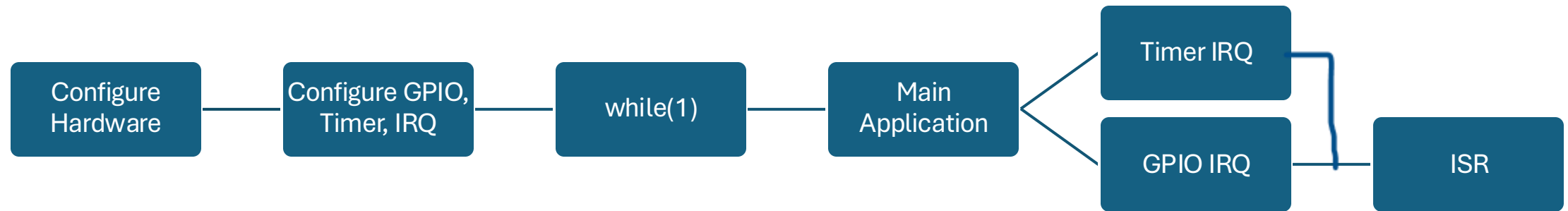


Interrupts: Responding to Events

- An interrupt allows hardware to temporarily stop normal program execution and tell the CPU that something needs attention



Putting it All Together



The Demo

Live Prototype of
a "Reaction-Time
Duel Game"

Walkthrough of
the Code

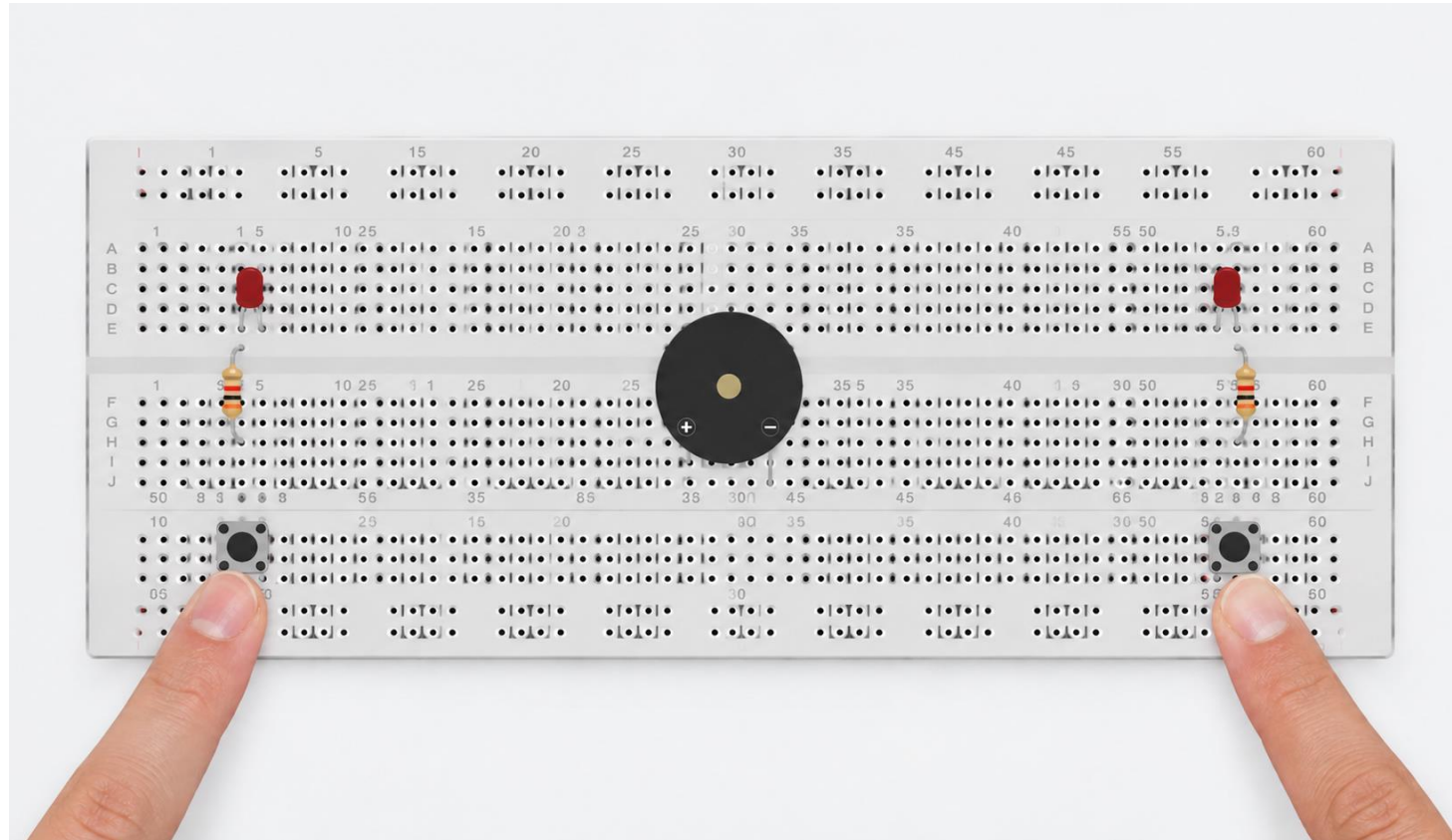
The reaction-time duel game

- A 2-player game to test who was the lowest reaction time.



The reaction-time duel game

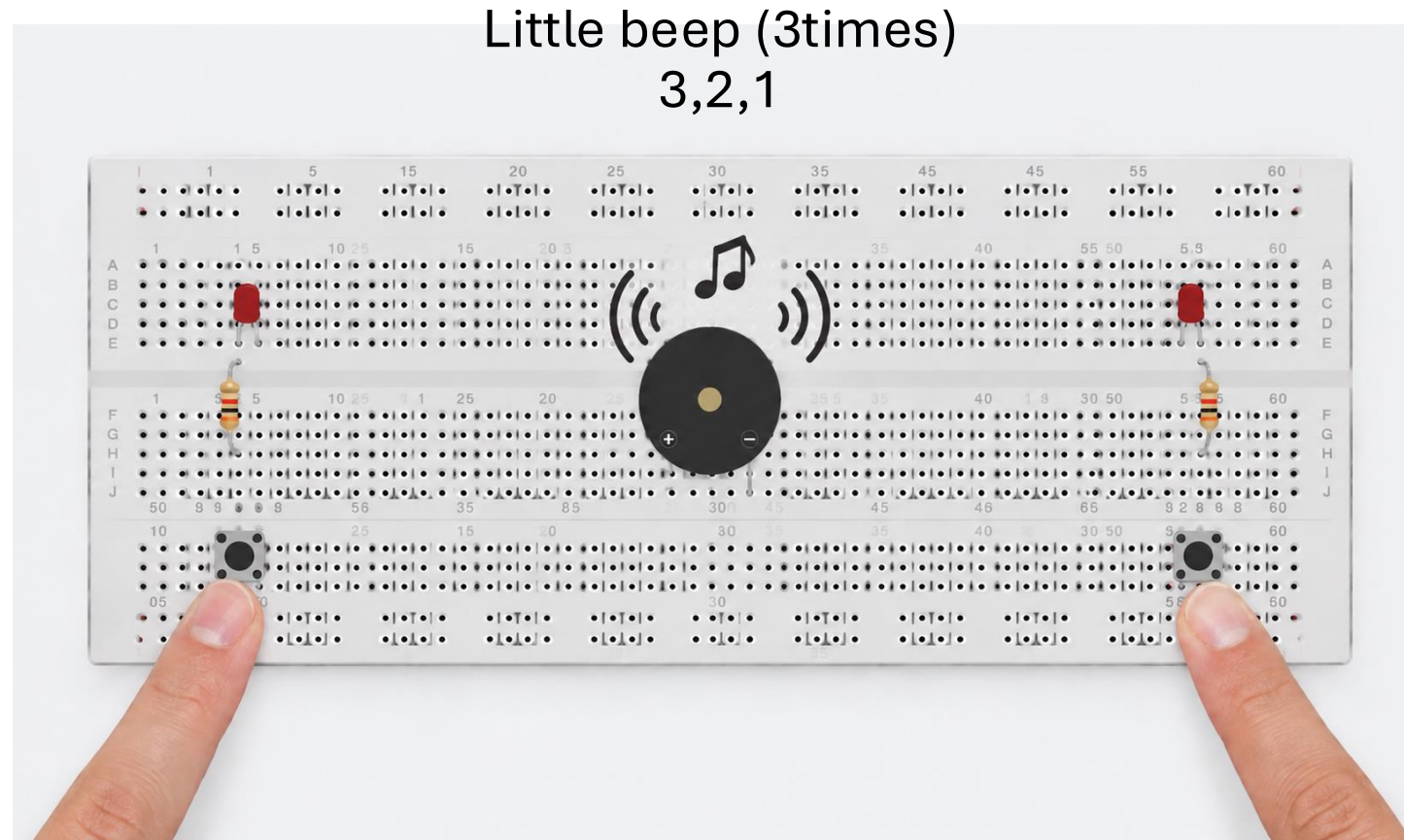
Player 1



Player 2

The reaction-time duel game

Player 1



Player 2

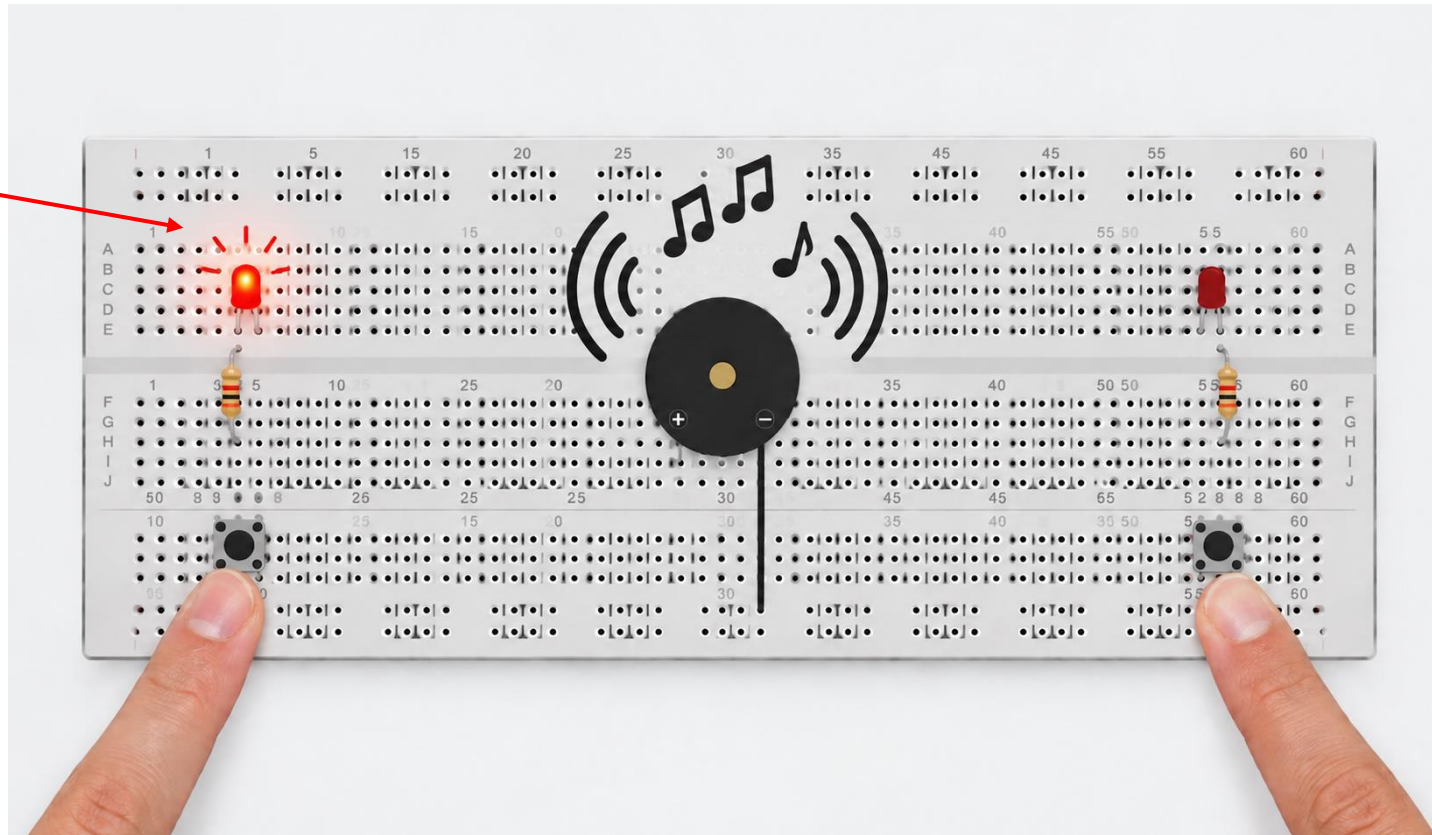
The reaction-time duel game

Big beep
PRESS THE BUTTON!

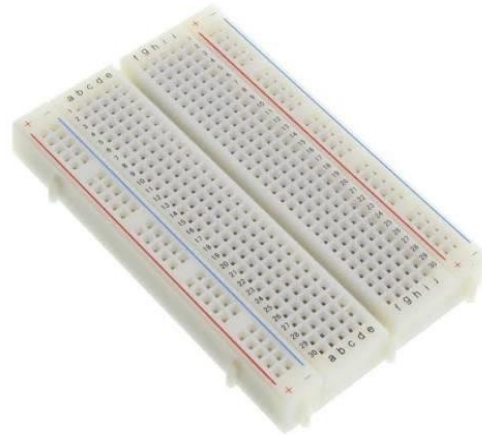
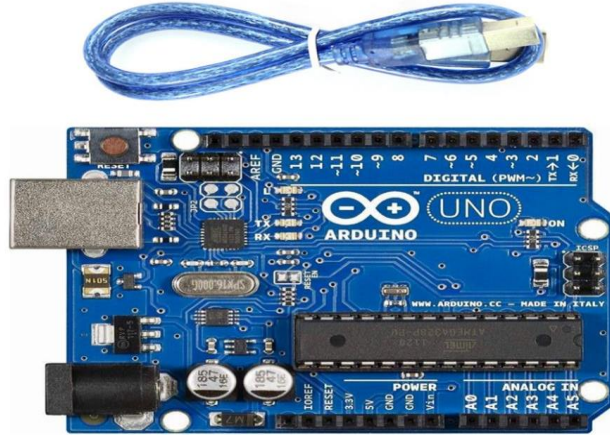
WINNER!

Player 1

Player 2



Hardware + components



2x



2x



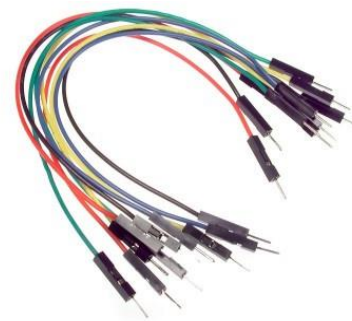
2x



2x



1x



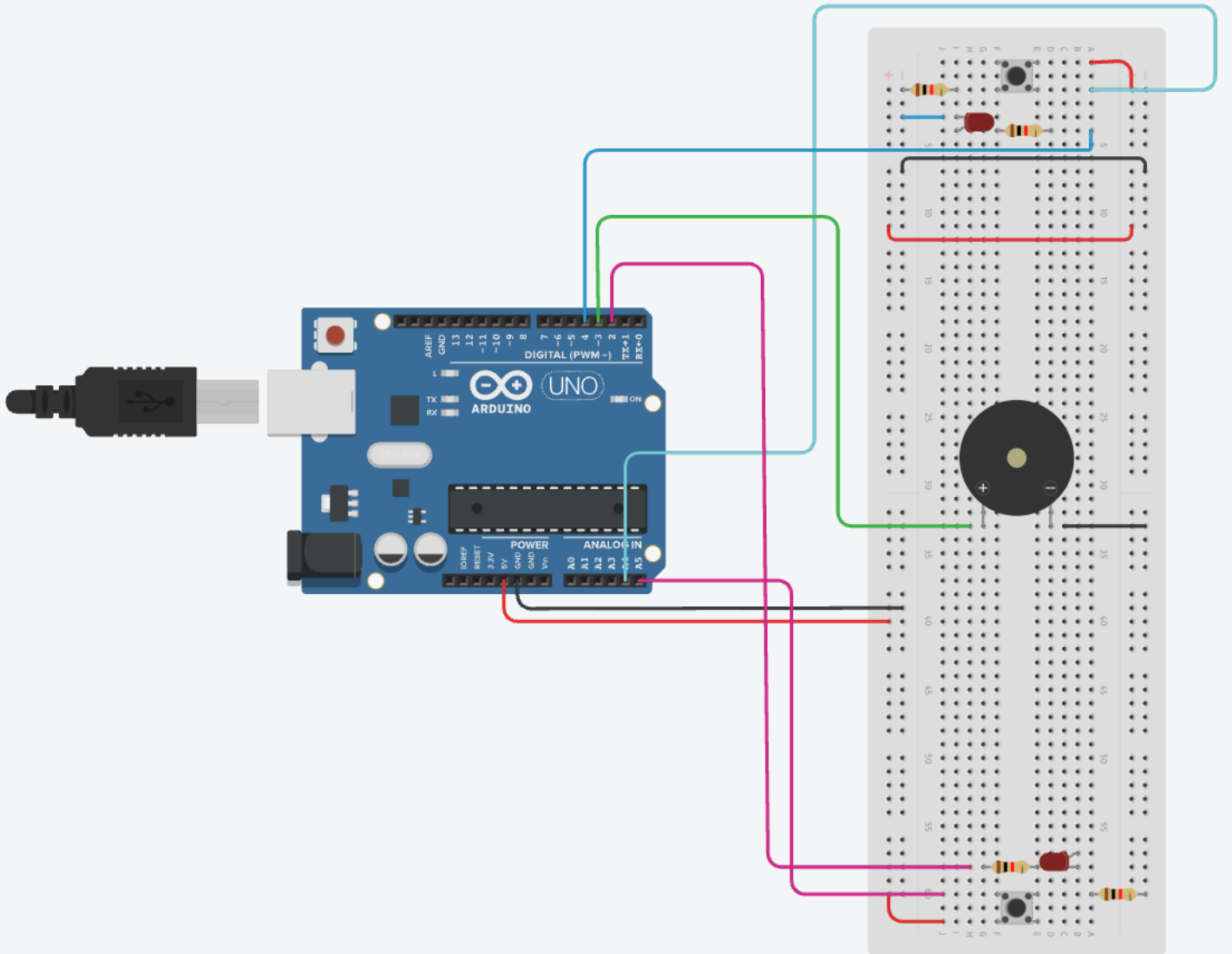
many

Components purposes

Component	Purpose
Arduino Uno	Controls the entire game
Push buttons	One button for each player
Leds	Indicates which player wins
Resistors 220-330 ohms	Protects the Leds from excessive current
Passive piezo buzzer	Produces the countdown and reaction sounds
breadboard	Connects everything without soldering
Jumper wires	Connects arduino to the components
USB cable	Connects arduino to the computer and provides power

Pin connections & Setup

Component	Arduino Pin
Player 1 button	A5
Player 2 button	A4
Player 1 LED	Digital Pin 2
Player 2 LED	Digital Pin 4
buzzer	Digital Pin 3
Button and LED grounds	GND
Buzzer ground	GND



Code

```
const int button1 = A5;
const int button2 = A4;
const int led1 = 2;
const int led2 = 4;
const int buzzer = 3;

void setup() {
  pinMode(button1, INPUT_PULLUP);
  pinMode(button2, INPUT_PULLUP);
  pinMode(led1, OUTPUT);
  pinMode(led2, OUTPUT);
  pinMode(buzzer, OUTPUT);
  randomSeed(analogRead(A0));
}

void loop() {
  // 1. Reset LEDs
  digitalWrite(led1, LOW);
  digitalWrite(led2, LOW);

  delay(1000);

  // 2. Countdown
  for (int i = 0; i < 3; i++) {
    tone(buzzer, 1000, 150);
    delay(1000);
  }

  // 3. Random delay
  delay(random(2000, 6000));

  // 4. Reaction signal
  tone(buzzer, 2000, 300);

  unsigned long startTime = millis();
```

```
  // 5. Wait for a button
  while (true) {

    if (digitalRead(button1) == HIGH) {
      unsigned long reactionTime = millis() - startTime;

      digitalWrite(led1, HIGH);

      tone(buzzer, 1000, 200);

      Serial.begin(9600);
      Serial.print("Player 1 wins! Reaction time: ");
      Serial.println(reactionTime);

      delay(5000);
      break;
    }

    if (digitalRead(button2) == HIGH) {
      unsigned long reactionTime = millis() - startTime;

      digitalWrite(led2, HIGH);

      tone(buzzer, 1500, 200);

      Serial.begin(9600);
      Serial.print("Player 2 wins! Reaction time: ");
      Serial.println(reactionTime);

      delay(5000);
      break;
    }
  }
}
```

Code

```
? const int button1 = A5;
? const int button2 = A4;
? const int led1 = 2;
? const int led2 = 4;
? const int buzzer = 3;
```

```
? void setup() {
?   pinMode(button1, INPUT_PULLUP);
?   pinMode(button2, INPUT_PULLUP);
?   pinMode(led1, OUTPUT);
?   pinMode(led2, OUTPUT);
?   pinMode(buzzer, OUTPUT);
?   randomSeed(analogRead(A0));
? }
```

```
void loop() {
// 1. Reset LEDs
digitalWrite(led1, LOW);
digitalWrite(led2, LOW);
```

```
delay(1000);
```

```
// 2. Countdown
for (int i = 0; i < 3; i++) {
tone(buzzer, 1000, 150);
delay(1000);
}
```

```
// 3. Random delay
delay(random(2000, 6000));
```

```
// 4. Reaction signal
tone(buzzer, 2000, 300);
```

```
unsigned long startTime = millis();
```



```
// 5. Wait for a button
while (true) {
```

```
if (digitalRead(button1) == HIGH) {
unsigned long reactionTime = millis() - startTime;
```

```
digitalWrite(led1, HIGH);
```

```
tone(buzzer, 1000, 200);
```

```
Serial.begin(9600);
Serial.print("Player 1 wins! Reaction time: ");
Serial.println(reactionTime);
```

```
delay(5000);
break;
}
```

```
if (digitalRead(button2) == HIGH) {
unsigned long reactionTime = millis() - startTime;
```

```
digitalWrite(led2, HIGH);
```

```
tone(buzzer, 1500, 200);
```

```
Serial.begin(9600);
Serial.print("Player 2 wins! Reaction time: ");
Serial.println(reactionTime);
```

```
delay(5000);
break;
}
```

Code syntax: variables

```
const int button1 = A5;  
  
digitalWrite(led1, HIGH);
```

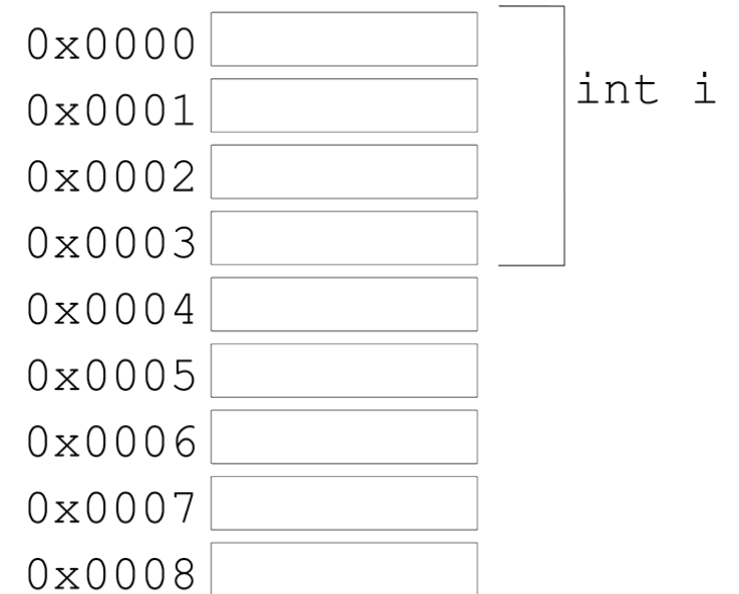
Variables

Used for storing information like numbers, characters, words.

Different data types take different # of bytes.

Example: int data type takes 4 bytes -> 4 consecutive memory addresses

Const just means we won't modify this variable later in this program, if we did, the compiler would let us know by flagging it as an error.



`digitalWrite(led1, HIGH);` is clearer than `digitalWrite(2,HIGH)`

Code syntax:

Data types

Data types

Used for storing different kinds of information like numbers, characters, words.

Different data types take different # of bytes.

```
int button1 = 1;  
char button2 = 'C';  
String word = "hello";  
Float decimal = 0.1;  
Bool sick = True
```

Type	Size (bits)	Size (bytes)	Range
char	8	1	-128 to 127
unsigned char	8	1	0 to 255
int	16	2	-2^{15} to $2^{15}-1$
unsigned int	16	2	0 to $2^{16}-1$
short int	8	1	-128 to 127
unsigned short int	8	1	0 to 255
long int	32	4	-2^{31} to $2^{31}-1$
unsigned long int	32	4	0 to $2^{32}-1$
float	32	4	$3.4\text{E}-38$ to $3.4\text{E}+38$
double	64	8	$1.7\text{E}-308$ to $1.7\text{E}+308$
long double	80	10	$3.4\text{E}-4932$ to $1.1\text{E}+4932$

Code syntax: Operators

Used to store, update, compare values from variables.

Example:

```
Int count = 0;
```

```
Count = count +1;
```

OPERATOR	TYPE	ASSOCIIVITY
() [] . ->		left-to-right
++ -- +- ! ~ (type) * & sizeof	Unary Operator	right-to-left
* / %	Arithmetic Operator	left-to-right
+ -	Arithmetic Operator	left-to-right
<< >>	Shift Operator	left-to-right
< <= > >=	Relational Operator	left-to-right
== !=	Relational Operator	left-to-right
&	Bitwise AND Operator	left-to-right
^	Bitwise EX-OR Operator	left-to-right
	Bitwise OR Operator	left-to-right
&&	Logical AND Operator	left-to-right
	Logical OR Operator	left-to-right
? :	Ternary Conditional Operator	right-to-left
= += -= *= /= %= &= ^= = <<= >>=	Assignment Operator	right-to-left
,	Comma	left-to-right

Code syntax: Functions

Used to build blocks of reusable code.

For example, a function that calculates the square power of a number.

```
Int pow(int a) {  
    return a * a;  
}
```

```
Int pow5 = pow(5);  
Cout << "the power of 5 is: " << pow5\n;
```

OPERATOR	TYPE	ASSOCIIVITY
() [] . ->		left-to-right
++ -- +- ! ~ (type) * & sizeof	Unary Operator	right-to-left
* / %	Arithmetic Operator	left-to-right
+ -	Arithmetic Operator	left-to-right
<< >>	Shift Operator	left-to-right
< <= > >=	Relational Operator	left-to-right
== !=	Relational Operator	left-to-right
&	Bitwise AND Operator	left-to-right
^	Bitwise EX-OR Operator	left-to-right
	Bitwise OR Operator	left-to-right
&&	Logical AND Operator	left-to-right
	Logical OR Operator	left-to-right
? :	Ternary Conditional Operator	right-to-left
= += -= *= /= %= &= ^= = <<= >>=	Assignment Operator	right-to-left
,	Comma	left-to-right

Lets pseudocode first

What do we want to do? In simple sentences:

1. Configure the buttons, leds, buzzer with arduino pins.
2. Start the led states as off
3. Make the buzzer beep three times to simulate a countdown.
4. Wait some random time
5. Make the beep signal to push the buttons
6. Start a timer
7. Read button press
8. Turn on winner's led
9. Calculate winner's time
10. Show winner's time
11. Repeat the game

Pseudocode -> source code

1. Configure the buttons, leds, buzzer with arduino

```
// Assign buttons, leds and buzzer to pins
```

```
Const int button1 = A5;  
Const int button2 = A4;  
Const int led1 = 2;  
Const int led2 = 4;  
Const int buzzer = 3;
```

```
// Assign pin modes
```

```
void setup() {  
    pinMode(button1, INPUT);  
    pinMode(button2, INPUT);  
  
    pinMode(led1, OUTPUT);  
    pinMode(led2, OUTPUT);  
  
    pinMode(buzzer, OUTPUT);  
  
    randomSeed(analogRead(A0));  
}
```

Pseudocode -> source code

2. Start the led states as off

```
// Start the LED states as off
```

```
digitalWrite(led1, LOW);  
digitalWrite(led2, LOW);
```

Pseudocode -> source code

3. Make the buzzer beep three times to simulate a countdown.

```
// Countdown  
  
for (int i = 0; i < 3; i++) {  
    tone(buzzer, 1000, 150);  
    delay(1000);  
}
```

Pseudocode -> source code

4. Wait some random time

```
// Wait some random time  
delay(random(2000, 6000));
```

Pseudocode -> source code

5. Push signal

```
// Make the beep signal to push the buttons
```

```
tone(buzzer,2000, 300);
```

Pseudocode -> source code

6. set a Timer

```
// Start a timer
```

```
unsigned long startTime = millis();
```

Pseudocode -> source code

7. Read who presses button first

```
// Read button press

while (true) {

    if (digitalRead(button1) == HIGH) {
        // Player 1 pressed the button
    }

    if (digitalRead(button2) == HIGH) {
        // Player 2 pressed the button
    }
}
```

Pseudocode -> source code

8. Turn on winner's led

```
// Turn on winner's LED
```

```
digitalWrite(led1, HIGH);
```



```
if (digitalRead(button1) == HIGH) {  
    // Player 1 pressed the button  
    ...  
}
```

Pseudocode -> source code

9. Calculate winner's time

```
// Calculate winner's time
```

```
unsigned long reactionTime = millis() - startTime;
```



```
if (digitalRead(button1) == HIGH) {  
    // Player 1 pressed the button  
    digitalWrite(led1, HIGH);  
    ...  
}
```

Pseudocode -> source code

10. Show winner's time

```
// Show winner's time  
Serial.print("Player 1 wins! Reaction time: ");  
Serial.print(reactionTime);  
Serial.println(" ms");
```



```
if (digitalRead(button1) == HIGH) {  
    // Player 1 pressed the button  
    digitalWrite(led1, HIGH);  
    unsigned long reactionTime = millis() - startTime;  
    ...  
}
```

Pseudocode -> source code

11. Repeat the game

```
digitalWrite(led1, LOW);  
...  
Serial.print("Player 1 wins! Reaction time: ");  
Serial.print(reactionTime);  
Serial.println(" ms");
```



```
void loop() {  
    // Step 1  
    ...  
    ...  
    ...  
    // Step 10  
}
```

Final code

```
const int button1 = A5;
const int button2 = A4;
const int led1 = 2;
const int led2 = 4;
const int buzzer = 3;

void setup() {
  pinMode(button1, INPUT_PULLUP);
  pinMode(button2, INPUT_PULLUP);
  pinMode(led1, OUTPUT);
  pinMode(led2, OUTPUT);
  pinMode(buzzer, OUTPUT);
  randomSeed(analogRead(A0));
}

void loop() {
  // 1. Reset LEDs
  digitalWrite(led1, LOW);
  digitalWrite(led2, LOW);

  delay(1000);

  // 2. Countdown
  for (int i = 0; i < 3; i++) {

    tone(buzzer, 1000, 150);

    delay(1000);

  }

  // 3. Random delay
  delay(random(2000, 6000));

  // 4. Reaction signal
  tone(buzzer, 2000, 300);

  unsigned long startTime = millis();
```

```
    // 5. Wait for a button
    while (true) {

      if (digitalRead(button1) == HIGH) {
        unsigned long reactionTime = millis() - startTime;

        digitalWrite(led1, HIGH);

        tone(buzzer, 1000, 200);

        Serial.begin(9600);
        Serial.print("Player 1 wins! Reaction time: ");
        Serial.println(reactionTime);

        delay(5000);
        break;
      }

      if (digitalRead(button2) == HIGH) {
        unsigned long reactionTime = millis() - startTime;

        digitalWrite(led2, HIGH);

        tone(buzzer, 1500, 200);

        Serial.begin(9600);
        Serial.print("Player 2 wins! Reaction time: ");
        Serial.println(reactionTime);

        delay(5000);
        break;
      }
    }
  }
}
```